

Creating a Private Subnet

AWS Cloud Portfolio Project

Roman Pillay

BSc Information Technology (Final Year)

romanpillay72@gmail.com

Amazon VPC

Private Subnet

Route Tables

Network ACLs

Zero-Trust Security

Introducing Today's Project!

What is Amazon VPC?

Amazon VPC (Virtual Private Cloud) is a logically isolated virtual network that you provision within the AWS cloud. It gives you complete control over your own virtual networking environment, including the selection of your own IP address ranges, the creation of subnets, and the configuration of route tables and network gateways. It is useful because it provides a highly secure and customizable foundation for running your cloud infrastructure. By isolating your resources from other AWS tenants, a VPC lets you define precise security boundaries like separating public web servers from private databases and gives you total control over how your applications connect to the internet, corporate data centers, or other cloud resources.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to build a secure, multi-tier network architecture from scratch that isolates sensitive data from the public internet. By creating a custom VPC, dividing it into distinct public and private subnets, and implementing dedicated route tables and network ACLs, I successfully established a hardened environment where public-facing web servers can communicate with the outside world while backend databases remain completely hidden and protected within a private zone.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is how completely unrestricted the default settings are when you first spin up a VPC. Seeing that a newly created private subnet automatically attaches to a main route table and a default network ACL that allows 100% of traffic in and out was a massive eye-opener. It really drove home the fact that AWS provides the raw building blocks, but it is

Private vs Public Subnets

The difference between public and private subnets is that a public subnet is explicitly configured with a route to an Internet Gateway, allowing resources within it to send and receive traffic directly to and from the public internet, whereas a private subnet lacks this route entirely, meaning its resources are completely isolated from external networks and can only communicate internally within the VPC.

Having private subnets are useful because they allow you to enforce a strict "zero-trust" security model within your infrastructure by completely hiding sensitive backend data and application logic from malicious scanners and automated internet threats. By placing databases and internal APIs in a private zone, you eliminate their public attack surface entirely, ensuring that even if a public-facing web server is compromised, your core data assets remain heavily guarded behind an additional layer of network isolation.

My private and public subnets cannot have the same IPv4 CIDR block. Every subnet within a VPC must be allocated its own unique, non-overlapping range of IP addresses so that the VPC's internal routers know exactly where to direct network traffic without any ambiguity or routing conflicts.

VPC

VPC ID

Create subnets in this VPC.

vpc-01f89158f29a42bec (NextWork VPC)

Associated VPC CIDRs

IPv4 CIDRs

10.0.0.0/16

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

Nextwork Private Subnet

The name can be up to 256 characters long.

Availability Zone

[Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Africa (Cape Town) / afs1-az2 (af-south-1b)

IPv4 VPC CIDR block

[Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.1.0/24

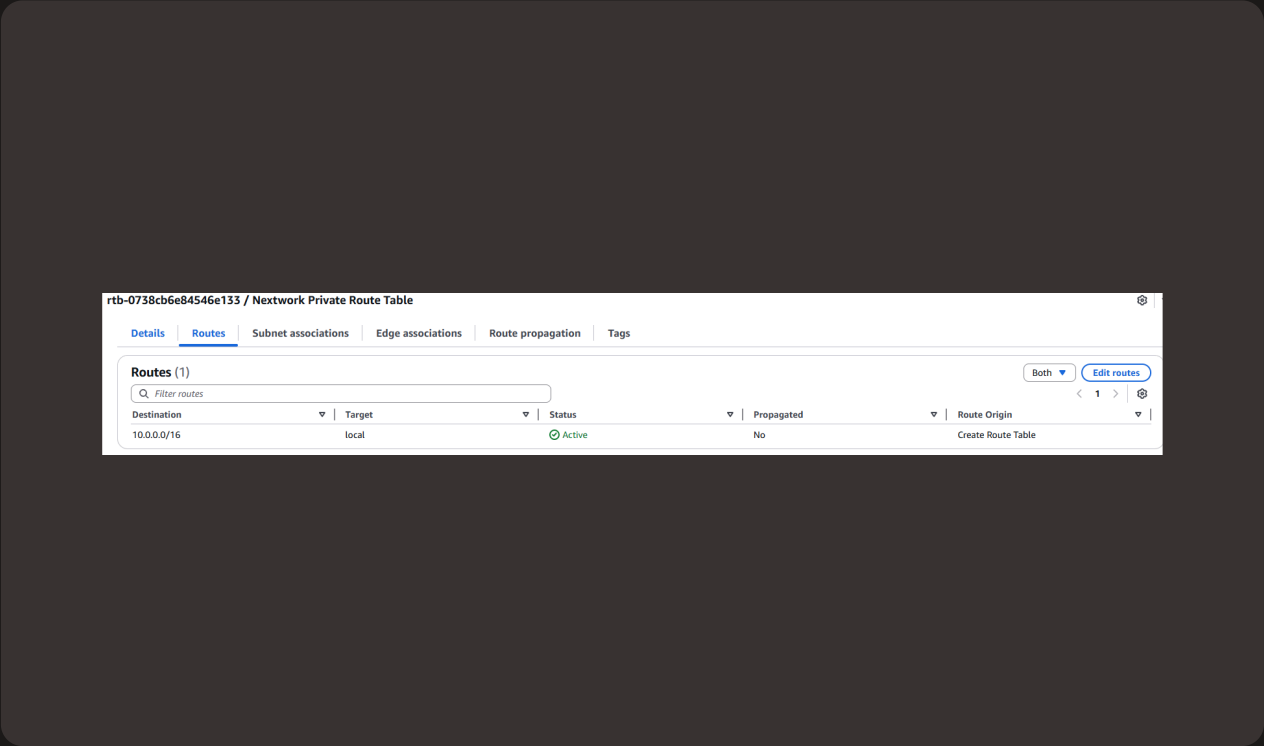
256 IPs

A dedicated route table

By default, my private subnet is associated with the VPC's main route table. When you create any new subnet in AWS, it is automatically linked to this default main route table until you explicitly create a custom route table and manually change its association.

I had to set up a new route table because leaving the private subnet attached to the main route table is a major security risk. If the main route table is ever modified to allow public internet access for other resources, the private subnet would automatically inherit those public routes as well, completely breaking its isolation and exposing sensitive backend data to the internet.

My private subnet's dedicated route table only has one inbound and one outbound rule that allows traffic to flow locally within the VPC's internal network range. Route tables don't actually use separate inbound and outbound rule lists (unlike Network ACLs), but their single target rule specifies that any data destined for an IP address inside our specific VPC CIDR block is routed to local, meaning resources can communicate seamlessly with each other across subnets while all traffic destined for the outside world is completely blocked.



A new network ACL

By default, my private subnet is associated with the VPC's default Network ACL. When you create a new subnet in AWS, it is automatically linked to this default NACL, which is configured to completely allow all inbound and outbound traffic (.0.0.0/0) until you create a custom NACL and manually reassociate the subnet to restrict it.

I set up a dedicated network ACL for my private subnet because the default Network ACL allows all traffic to pass through unchecked, which poses a major security risk for sensitive backend infrastructure. By creating a dedicated, custom NACL, I can enforce a strict "least privilege" security posture right at the subnet boundary allowing me to specifically blacklist or whitelist traffic types based on the exact needs of my private resources, completely independent of the rules governing my public subnet.

My new network ACL has two simple rules - one inbound rule and one outbound rule that lock down the subnet's perimeter. The inbound rule is configured to allow traffic only from within my VPC's specific IP address range (our internal CIDR block) on necessary ports, while the outbound rule is set to allow traffic to return back to that same internal VPC network. Because Network ACLs are stateless, both rules must be explicitly defined to ensure that internal resources can successfully send requests and receive responses, while any traffic originating from outside the VPC is completely blocked.

